TÉCNICO LISBOA

Lab 3: Local storage

CMU 2024/25

Learning objectives

In this lab you will:

- Part I
  - Structured data with Android Room
  - Shared preferences with Data Store
  - How to store files in the Android File System
- Part II
  - Android Backends – Firebase and Supabase

TÉCNICO LISBOA

Part 1

Android local storage

Local Storage in Android

- Why local storage is important in mobile apps
- Types of data stored locally: user data, app settings, cache, etc.
- Android offers several APIs for persistence

Local Storage Options

- **Room** – Database abstraction over SQLite
- **DataStore** – For for small key-value or typed data (example: the Settings app)
- **File System** – Direct file I/O for custom data formats
- Choosing the right tool depends on **data structure**, **size**, and **complexity**

Android Room – What & Why

- Room is an abstraction layer over SQLite
- Provides compile-time query verification
- Uses annotations and DAO (Data Access Objects)
- Good for structured, relational data

Android Room – Key Components

- @Entity – Maps to a SQLite table
- @Dao – Defines database access methods
- RoomDatabase – Abstract class, app’s DB holder
- Example:

```
@Query("SELECT * FROM users") fun getAll(): List<User>
```

DataStore – Introduction

- Jetpack library for key-value or typed object storage
- Replaces SharedPreferences
- Two types: Preferences DataStore and Proto DataStore
- Asynchronous, coroutine-friendly, transactional

DataStore – Use Cases & Example

- Preferences DataStore for settings/preferences
  - Key-value access
- Proto DataStore for typed objects using protobuf schema
  - Example: `UserPreferences.proto`
- Safer and more scalable than SharedPreferences

File System Storage

- Use standard file I/O to read/write files
- Internal vs External storage:
  - Internal is private to app
  - External may require permissions
- Good for binary, media, or custom formats

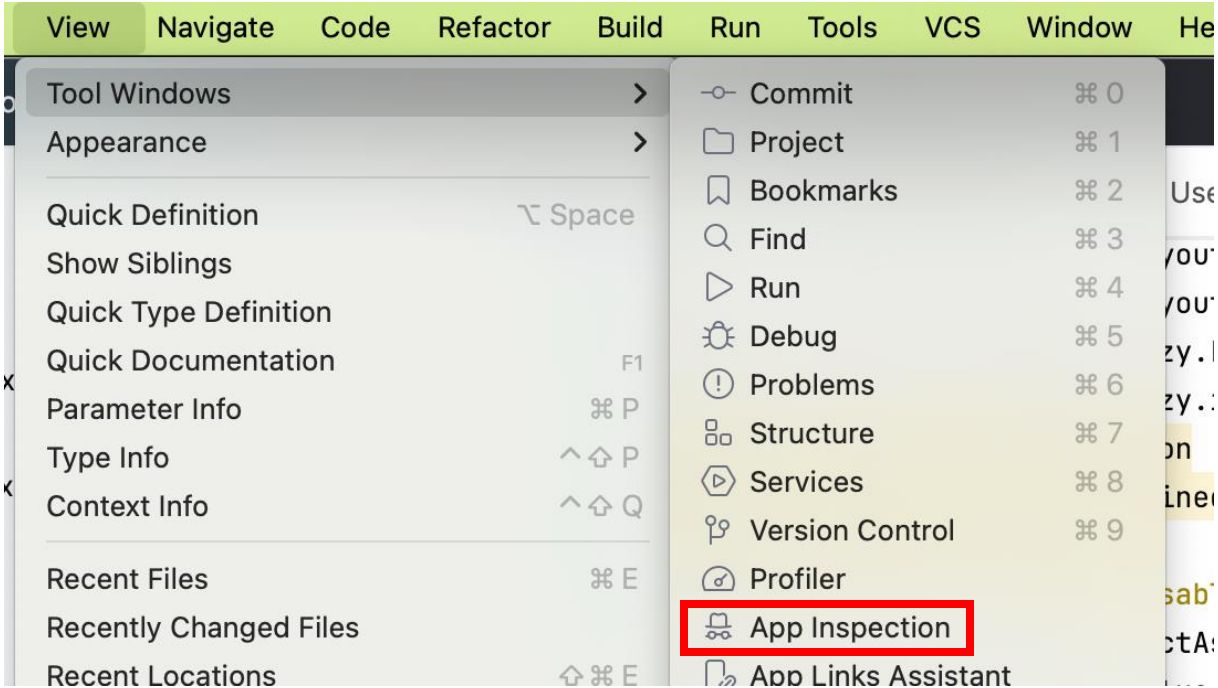
Android Local storage – Best option

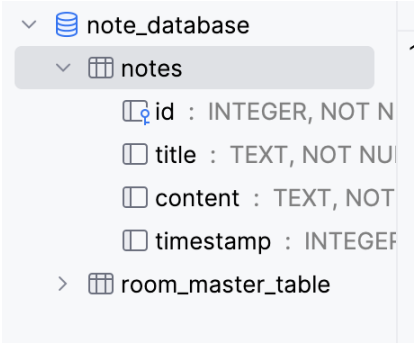
Feature	Room	DataStore	File System
Structured Data	✅	❌ (Proto optional)	❌
Transactional	✅	✅	❌
Binary/Media	❌	❌	✅
Async support	✅ (Coroutines)	✅ (Coroutines)	❌ (manual)

Summary and Best Practices

- Use Room for relational data needs
- DataStore is ideal for user settings/preferences
- File system fits custom or media-heavy data
- Always consider encryption, backup, and data migration
- Choose based on **data structure**, **size**, **complexity**, and **access pattern**

How to debug the database?





TÉCNICO LISBOA

Part 2

Backends for Android

Why Backends Matter in Android Development

- Enable dynamic content and real-time features
- Store user data remotely (e.g., cloud databases)
- Authentication, push notifications, analytics
- Three common backend options:
  - Custom Server via REST API
  - Firebase (Google)
  - Supabase (open source)

Server Through REST API

- Create your own backend using Node.js, Python, Java, etc.
- Expose endpoints over HTTP using REST
- Android communicates via Retrofit, Volley, or HttpURLConnection
- Full control over business logic and data

REST API – Architecture & Example

- Client (Android) → HTTP Request → Server (REST API) → Database
- Requires hosting, security, scaling management
- Typical stack: Node.js + Express + PostgreSQL
- Example:

```
@GET("users/{id}")
suspend fun getUser(@Path("id") id: Int): User
```

Firebase – Overview

- Google’s mobile backend-as-a-service (MBaaS)
- Offers real-time database, Firestore, Auth, Cloud Functions, Storage, Analytics
- SDKs directly integrate with Android
- Freemium pricing, generous free tier

Firebase – Core Services

- **Authentication** – Email, Google, Facebook, etc.
- **Cloud Firestore** – NoSQL database with real-time updates
- **Realtime Database** – Sync data in real time
- **Cloud Storage** – Store and serve media files
- **Cloud Messaging** – Push notifications

Supabase – Overview

- Open source alternative to Firebase
- Built on PostgreSQL + RESTful + GraphQL + Realtime
- Includes: Auth, Database, Storage, Functions
- Self-hostable or use Supabase Cloud

Supabase – Features & Integration

- **PostgreSQL** – Relational DB with SQL support
- **Supabase Auth** – Email, OAuth, magic links
- **Storage** – S3-compatible
- **Client SDKs** for Android, Kotlin (via REST or GraphQL)
- Can be hosted or used on your own server

Summary

Feature	REST API	Firebase	Supabase
Hosting Needed	✅	❌	❌ (optional)
Realtime Data	❌ (custom)	✅	✅
SQL Support	✅	❌ (NoSQL)	✅ (PostgreSQL)
Custom Logic	✅ (full)	Limited	✅ (Edge Functions)
Open Source	✅	❌	✅

What’s the best choice?

- **REST API** – Full control, best for complex/custom logic
- **Firebase** – Quick setup, real-time sync, great for MVPs and apps with Firebase services
- **Supabase** – Open source, SQL, flexible alternative to Firebase
- Consider your app’s:
  - Data structure and sync needs
  - Hosting and scaling ability
  - Feature set and vendor lock-in risk

TÉCNICO LISBOA